SIMATS ENGINEERING
Assignment

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO1: Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices.

CO2: Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system.

CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.

Assignment Weightage: 100%

QUESTIONS: 1 **Total Marks:100**

Annexure A

Assignment

Code of Conduct:

*I **GLADIES A 192520026** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: GLADIES A 192520026

Course Code & Name

CSA11 – Object Oriented Analysis and Design

Assignment Title	**Object-Oriented Analysis and UML-Based Design of an Autonomous Drone-Based Disaster Response and Rescue System (ADDRRS)**
Course Outcome(s) – CO	**CO1:** Analyse Object-Oriented Systems Development concepts, methodologies, life cycle models, MDD, and Agile practices for developing a real-world software solution. **CO2:** Apply UML modelling techniques to represent system structure, interactions, workflows, behaviour, components, and deployment architecture.

	CO3: Analyse objects, classes, attributes, methods, and relationships using Use Case Modelling, Object Analysis, Classification, and relationship identification techniques.
Bloom's Taxonomy Level	BL3 – Apply, BL4 – Analyse, BL5 – Evaluate, BL6 – Create
SDG Mapping (SDG 1–17)	**SDG 3 – Good Health and Well-being** **SDG 9 – Industry, Innovation and Infrastructure** **SDG 11 – Sustainable Cities and Communities** **SDG 13 – Climate Action**

Problem Statement

Analyse and develop an **Object-Oriented Analysis and Design model for an Autonomous Drone-Based Disaster Response and Rescue System (ADDRRS)** that supports emergency teams in locating affected people, assessing disaster zones, coordinating rescue operations, and monitoring emergency situations.

The system shall allow authorized emergency personnel to register disaster incidents, define affected zones, assign drones, monitor drone status, and manage rescue missions. Autonomous drones equipped with cameras and sensors shall collect information about disaster locations and transmit images, videos, environmental readings, and location information to the central command system.

The system shall maintain information about **drones, operators, rescue missions, disaster zones, victims, emergency teams, sensor data, alerts, and resources**. The system shall support route planning, mission assignment, real-time drone monitoring, emergency notifications, and mission completion reporting.

Working from this scenario, students shall identify the major actors and use cases and develop **Use Case, Class, Sequence, Activity, State Transition, Package, Component, and Deployment Diagrams**. Students shall identify object responsibilities and evaluate suitable design principles and patterns for achieving low coupling, high cohesion, flexibility, and maintainability.

Constraints

The design shall clearly identify the **system boundary, emergency actors, autonomous components, objects, classes, attributes, methods, and relationships**. All UML diagrams shall maintain consistency with the identified use cases and system behaviour.

The model shall explicitly consider **real-time communication, autonomous operations, failure handling, security, reliability, scalability, and responsibility assignment**. Design axioms and suitable design patterns shall be applied wherever appropriate.

The final submission shall include assumptions, constraints, traceability between use cases and classes, and justification for major design decisions.

Tools Can Be Used

- StarUML
- Visual Paradigm
- Draw.io
- PlantUML
- Papyrus Tools
- ArgoUML
- Enterprise Architect
- Lucidchart

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding, Object Identification & Requirements Analysis (CO1)	**10**	Clearly analyses the problem domain and accurately identifies all major actors, objects, system responsibilities, and requirements from the given scenario.	Identifies most actors, objects, and requirements with minor gaps or inaccuracies.	Identifies basic actors and objects but analysis is incomplete or contains some inconsistencies.	Poor understanding of the problem; major actors, objects, and requirements are missing or incorrectly identified.
Use Case Modelling & Actor Identification (CO2)	**10**	Correctly identifies all relevant actors and use cases; Use Case Diagram clearly represents system boundary, relationships, and interactions.	Most actors and use cases are correctly identified with minor modelling errors.	Basic actors and use cases are identified, but relationships or system boundary need improvement.	Actors and use cases are poorly identified; diagram is incomplete or inconsistent.
Class Identification, Attributes, Methods & Relationships (CO2/CO3)	**15**	Classes, attributes, methods, associations, aggregation/composition, inheritance, and dependencies are accurately identified and logically justified.	Class structure is mostly correct with minor errors in attributes, methods, or relationships.	Basic classes and relationships are identified but lack detail or contain several modelling issues.	Classes, attributes, methods, or relationships are missing, inappropriate, or poorly structured.
UML Behavioural & Interaction Modelling (CO2)	**15**	Sequence, Activity, and State Transition Diagrams accurately represent system behaviour and are fully consistent with the identified use cases and classes.	Behavioural diagrams are mostly correct with minor inconsistencies.	Basic behavioural diagrams are provided but some interactions, activities, or states are incomplete.	Behavioural diagrams are missing, unclear, or inconsistent with the proposed system.
UML Architectural Modelling & Design Evaluation (CO3)	**15**	Package, Component, and Deployment Diagrams clearly represent modular structure, system components, dependencies, and deployment environment with strong justification.	Architectural diagrams are mostly correct with minor omissions or modelling errors.	Basic architectural representation is provided but lacks clarity or sufficient justification.	Architectural diagrams are incomplete, inappropriate, or inconsistent with the system design.

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Object Responsibilities, Design Principles, Axioms & Patterns (CO3)	**15**	Responsibilities are appropriately assigned; design axioms, cohesion, coupling, encapsulation, abstraction, and suitable design patterns are clearly justified and effectively applied.	Design principles and patterns are appropriately applied with minor limitations in justification.	Some design principles are identified, but application or justification is limited.	Design principles, axioms, responsibilities, or patterns are absent or incorrectly applied.
Consistency, Traceability & Overall Design Quality (CO3)	**10**	All UML models are mutually consistent, requirements are traceable to use cases/classes, and the overall design is complete, feasible, scalable, and maintainable.	Design is largely consistent and traceable with minor gaps.	Some consistency and traceability are demonstrated, but important links are missing.	Major inconsistencies exist between requirements, use cases, classes, and UML diagrams.
Reflection, SDG Relevance & Learning Outcomes	**10**	Provides insightful reflection on design decisions, challenges, OOAD concepts learned, and clearly establishes the relevance of the selected SDGs.	Provides good reflection with reasonable connection to design decisions and SDGs.	Reflection is present but generic with limited discussion of learning and SDG relevance.	Reflection is missing, superficial, or unrelated to the assignment and SDGs.
Total	**100**				

Deliverables

To receive full credit, each submission must include the following:

1. **Problem Analysis and Requirement Summary**
2. **Actor Identification**
3. **Use Case Descriptions**
4. **Use Case Diagram**
5. **Object and Class Identification**
6. **Class Diagram**
7. **Sequence/Interaction Diagram(s)**
8. **Activity Diagram**
9. **State Transition Diagram**
10. **Package Diagram**
11. **Component Diagram**
12. **Deployment Diagram**
13. **Object Responsibility Analysis**
14. **Design Axioms/Principles and Design Pattern Justification**
15. **Assumptions and Constraints**
16. **Requirement-to-Use Case/Class Traceability**
17. **Implementation/Prototype and Results**
18. **Reflection on Design Decisions and Learning Outcomes**
19. **GitHub Upload**

1. Problem Analysis and Requirement Summary

1.1 Restated Problem Statement

Large-scale disasters such as earthquakes, floods, and gas leaks can occur suddenly and leave an imprint of death and destruction. It is impossible for ground rescuers to reach all of the affected areas in time, especially when they are cut off from the road by landslide debris. The Autonomous Drone-Based Disaster Response and Rescue System (ADDRRS) is an object-oriented software system that is designed to help emergency coordinators respond to disasters by allowing them to file incident reports and notify rescue teams. It works by dispatching a fleet of autonomous drones to the affected area, where they can survey it with onboard video cameras and sensors. The drones can navigate through the target area with minimal human input while transmitting imagery and videos to the command center for emergency coordinators to analyze. It empowers field rescue teams by relaying information about detected victims and allows emergency coordinators to keep track of all rescue operations.

The problem is to design the software system according to the requirements and represent it in various UML diagrams. It must be noted that all UML artifacts should be consistent with one another, with each use case connected to a class and all components synchronized. Moreover, every element should have an appropriate name without using any abbreviations that may differ between actors and classes.

1.2 Functional Requirements

Final requirements specification for the rescue coordination system

These requirements were developed by the team to standardize the operations of the new system, which will be utilized during emergency situations. It is critical to ensure that all components are well-explained while identifying specific user demands. In this document, specific standards are established for the created software to operationally comply with the suggested instructions. The requirements are prioritized based on the significance and complexity of each component.

•FR1: The system shall allow an authenticated Emergency Coordinator to register a new disaster incident with type, severity, location, and time of report

•FR2: The system shall allow the coordinator to define one or more affected zones associated with an incident with zone boundary and estimated risk level

•FR3: The system shall allow assignment of one or more available drones to a rescue mission targeting a specific zone

•FR4: The system shall plan and optimize a flight route for each assigned drone based on the zone boundary and known obstacles

•FR5: The system shall allow a Drone Operator to monitor real-time drone status (battery, location, health) and override navigation when required

•FR6: Each drone shall autonomously capture images, video, and environmental sensor readings and transmit them to the central command system

•FR7: The system shall automatically raise an alert when a drone's sensor data indicates a probable victim, hazard, or structural failure

•FR8: The system shall notify relevant Rescue Team Members when an alert is raised, indicating the location and nature of the detection

•FR9: Rescue Team Members shall be able to update the status of a tracked victim and report on rescue progress

•FR10: The system shall allow coordinators to allocate and release resources (medical kits, vehicles, personnel) against a mission

•FR11: The system shall record drone faults or failures, triggering a fail-safe return-to-base procedure

•FR12: The system shall generate a mission completion report summarizing the duration, findings, resource usage, and outcomes

1.3 Non-Functional Requirements

- NFR1: Timeliness: telemetry and other data sent from the drone to the control system and operators should be delivered with a minimum delay (preferably lower than 3 seconds in case of no overload).
- NFR2: Autonomy: drones should be capable of performing navigation and sensing tasks by themselves, requiring no human input except for explicit commands and error resolution.
- NFR3: Fault tolerance: in case of drone failure and/or communication disruption, a set of predefined actions should be taken to resolve the issue without any operator intervention.
- NFR4: Security: authorization should be required for all coordinators, operators, and rescue team members, as well as have command channels protected from unauthorized access attempts.
- NFR5: Reliability: in case of a single drone failure, the system should continue operating and not disrupt the mission.
- NFR6: Scalability: the system should be designed in a way that allows it to handle an increasing number of drones, incidents, and zones without architectural changes.
- NFR7: Responsibility: responsibility between classes should be well-defined, with no class having global access to other classes' data and operations (no god classes).
- NFR8: Usability: operators should not require any specific training to use the system as it should be intuitive and provide all the required information (alerts, drone status, etc.) in dashboards and mobile interfaces.

2. Actor Identification

Actors have been identified by walking through the problem statement and constraints and asking, for each noun phrase describing a role, whether it initiates a use case, receives information from the system under development, or is an external system that the ADDRRS must interface with. This resulted in three human primary actors, one autonomous system actor, and two supporting/external actors.

Actor	Type	Description
Emergency Coordinator	Primary (Human)	Registers incidents, defines affected zones, assigns drones and resources to missions, and reviews mission reports. Has the broadest administrative view of the system.
Drone Operator	Primary (Human)	Monitors drone telemetry, plans/approves routes, and takes manual control if a drone needs override or recovery.
Rescue Team Member	Primary (Human)	Receives victim/hazard alerts, tracks victims in the field, and reports on rescue mission progress.
Autonomous Drone	Primary (System)	Acts on the system's behalf once dispatched: navigates, captures sensor data, and raises alerts without step-by-step human instruction.
Sensor Subsystem	Supporting (System)	Onboard cameras and environmental sensors that feed raw readings into the drone's data-capture use cases.
Notification Service	External System	Third-party SMS/push notification gateway used to deliver alerts to rescue teams and coordinators.

3. Use Case Descriptions

3.1 Use Case Summary

Use Case	Primary Actor(s)	Brief Description
Register Disaster Incident	Emergency Coordinator	Capture a new incident's type, severity, location, and time.
Define Affected Zone	Emergency Coordinator	Draw and classify a geographic zone associated with an incident.
Assign Drone to Mission	Emergency Coordinator, Drone Operator	Select an available drone and bind it to a rescue mission.
Plan Route	Drone Operator, Drone	Compute a flight path across a zone, avoiding known obstacles.
Monitor Drone Status	Drone Operator	View live telemetry — battery, location, health — for active drones.
Collect Sensor Data	Drone, Sensor Subsystem	Capture imagery, video, and environmental readings during flight.
Transmit Media & Environmental Data	Drone	Send captured data from the drone to the command system.
Manage Rescue Mission	Emergency Coordinator, Rescue Team Member	Create, update, and close a rescue mission's lifecycle.
Track Victim	Rescue Team Member	Record and update the location/condition of a detected victim.
Send Emergency Notification	Notification Service	Deliver alerts to the relevant rescue team or coordinator.

Use Case	Primary Actor(s)	Brief Description
Manage Resource	Emergency Coordinator	Allocate or release medical kits, vehicles, or personnel.
Generate Mission Report	Emergency Coordinator	Summarise a completed mission's duration, findings, and outcome.
Handle Drone Failure/Alert	Drone, Drone Operator	Detect malfunction and trigger a fail-safe or escalate an alert.
Authenticate User	All human actors	Verify credentials before granting role-based system access.

3.2 Fully Dressed Use Case Specifications

Use Case: Assign Drone to Mission

Field	Description
Actors	Emergency Coordinator (primary), Drone Operator, Autonomous Drone
Preconditions	An affected zone has been defined; at least one drone has status "Idle" and battery above the minimum operating threshold.
Postconditions	A RescueMission object exists in "Assigned" state with a bound Drone and planned Route.
Main Flow	1. Coordinator selects an affected zone requiring survey. 2. System lists available drones filtered by battery level and proximity. 3. Coordinator selects a drone and confirms mission priority. 4. System creates a RescueMission and invokes Plan Route (include). 5. Drone Operator reviews and approves the generated route. 6. System updates drone status to "Assigned" and notifies the operator.
Alternate Flow	3a. No drone is available: system places the request in a queue and notifies the coordinator when a drone frees up.
Exception Flow	5a. Operator rejects the route: system returns to step 4 for re-planning with updated constraints.

Use Case: Track Victim (with Send Emergency Notification)

Field	Description
Actors	Rescue Team Member (primary), Autonomous Drone, Notification Service
Preconditions	A drone survey has produced sensor data classified as a possible victim detection.
Postconditions	A Victim record exists with last known location and condition; the assigned rescue team has been notified.
Main Flow	1. Drone's sensor data triggers a victim-detection alert (extend of Collect Sensor Data). 2. System creates or updates a Victim record with location and estimated condition. 3. System invokes Send Emergency Notification (include) to the nearest rescue team. 4. Rescue Team Member acknowledges the alert and proceeds to the location. 5. Rescue Team Member updates victim condition and rescue status upon arrival.
Alternate Flow	1a. Detection confidence is below threshold: system logs the reading without creating an alert.

Field	Description
Exception Flow	3a. Notification Service is unreachable: system retries delivery and escalates to the coordinator's dashboard as a fallback.

Use Case: Register Disaster Incident

Field	Description
Actors	Emergency Coordinator (primary)
Preconditions	The coordinator is authenticated (include Authenticate User).
Postconditions	A new DisasterIncident record exists with status "Open".
Main Flow	1. Coordinator logs in (include Authenticate User). 2. Coordinator enters incident type, severity, location, and time. 3. System validates the entry and creates a DisasterIncident with status "Open". 4. System prompts the coordinator to define at least one affected zone.
Alternate Flow	2a. Coordinator saves an incomplete draft: system stores it with status "Draft" for later completion.
Exception Flow	1a. Authentication fails: system denies access and logs the attempt.

3.3 Use Case Diagram

The diagram below depicts all identified actors and use cases together with include/extend relationships. «include» is used where a sub-behaviour is always required (e.g. Assign Drone to Mission always includes Plan Route), and «extend» is used for conditional behaviour that only fires under specific conditions (e.g. Handle Drone Failure/Alert only extends Monitor Drone Status when a fault actually occurs).

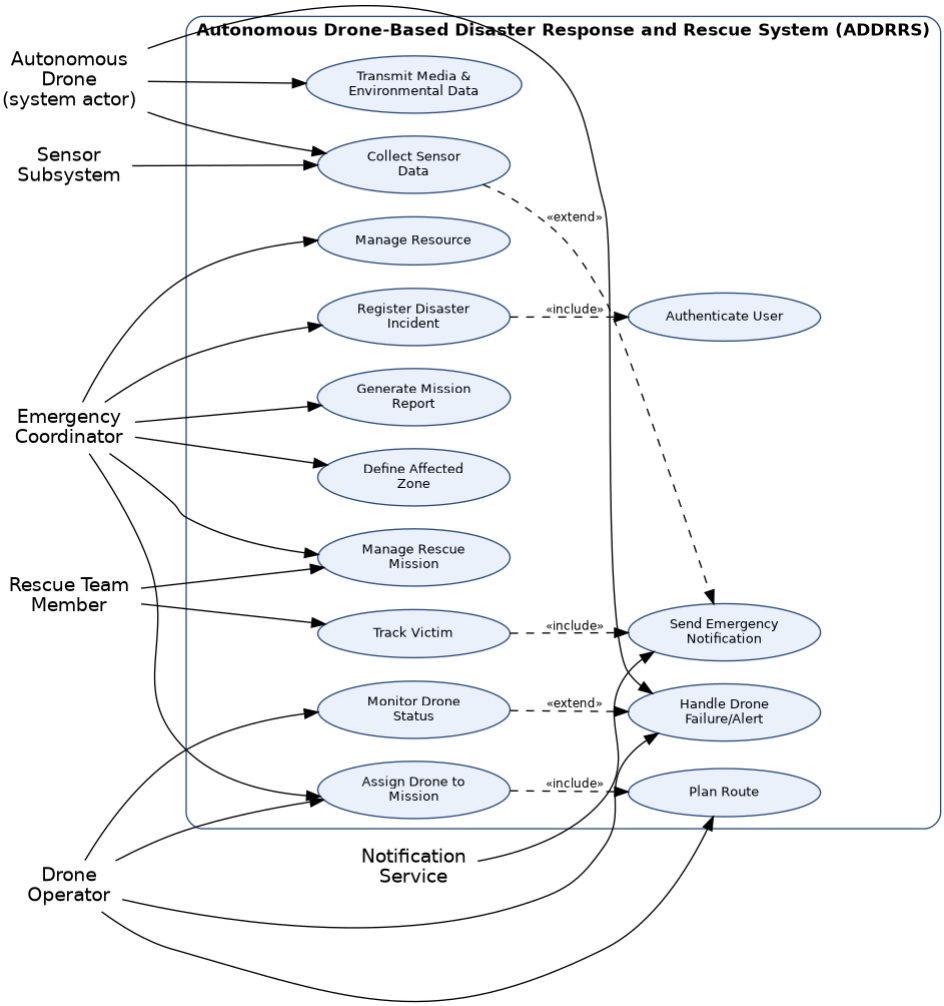

Figure 4.1 — Use Case Diagram for ADDRRS

5. Object and Class Identification

Classes were identified using noun-phrase analysis of the problem description and the use case specifications, followed by a filtering process to eliminate duplicates and purely descriptive terms. The remaining candidates were assigned the analysis-level stereotype entity (maintains persistent data), boundary (handles interaction with an actor), or control (coordinates a use case's logic).

Class	Stereotype	Key Attributes	Responsibility
DisasterIncident	Entity	incidentId, type, severity, location, reportedTime, status	Represent a reported disaster and its lifecycle.
AffectedZone	Entity	zoneId, boundary, riskLevel, population	Represent the geographic area a mission targets.
RescueMission	Control/Entity	missionId, objective, startTime, status, priority	Coordinate drone assignment, team assignment, and mission lifecycle.
Drone	Entity	droneId, model, batteryLevel, currentLocation, status	Represent an autonomous aerial unit and its operational state.

Class	Stereotype	Key Attributes	Responsibility
Operator (abstract)	Entity	operatorId, name, role, contact	Common base for all human roles that authenticate into the system.
SensorData	Entity	dataId, timestamp, readingType, value, mediaUrl	Hold a single captured reading or media item from a drone.
Victim	Entity	victimId, lastKnownLocation, condition, status	Track a detected person requiring rescue.
Alert	Control/Entity	alertId, alertType, severity, timestamp	Represent a triggered notification-worthy event.
Resource	Entity	resourceId, type, quantity, availability	Represent an allocatable asset (kit, vehicle, personnel).
Route	Entity	routeId, waypoints, distance, estimatedTime	Represent a planned flight path for a drone.
CommandCenter	Control	centerId, activeMissions	Coordinate cross-mission monitoring; implemented as a Singleton.
NotificationService	Boundary	serviceId, channel	Interface to the external SMS/push notification gateway.

6. Class Diagram

This is represented by a class diagram which models the above objects along with their attributes, operations and relationships. There is only one application of generalization – DroneOperator, EmergencyCoordinator and RescueTeamMember are all specialized versions of the abstract class Operator and implement similar login/logout operations but override certain operations specific to their roles. There is an application of composition (a filled diamond) in cases where the component object cannot outlive the owner object – e.g., RescueMission has Resource objects as its components. Aggregation and association are used in cases where objects are independent of each other.

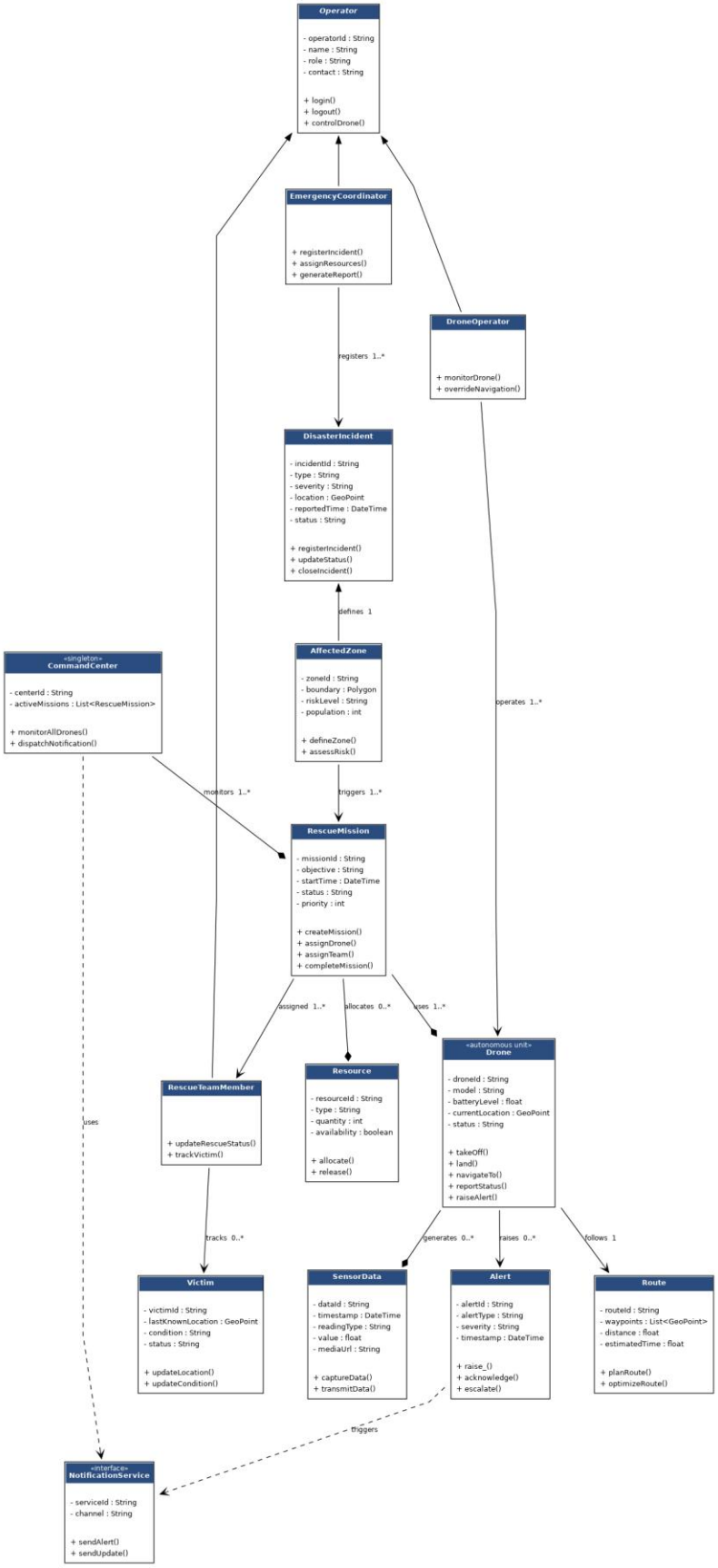

Figure 6.1 — Class Diagram for ADDRRS

6.1 Relationship Summary

Relationship	Multiplicity	Type	Rationale
EmergencyCoordinator → DisasterIncident	1 coordinator : 0..* incidents	Association	A coordinator registers many incidents over time.
DisasterIncident → AffectedZone	1 : 1..*	Composition	A zone has no meaning outside the incident that defines it.
RescueMission → Drone	1..* : 1..*	Association	A mission can use several drones; a drone can serve multiple missions over its life.
RescueMission → Resource	1 : 0..*	Composition	Allocated resources belong to exactly one active mission at a time.
Drone → SensorData	1 : 0..*	Composition	Sensor readings are generated and owned by the capturing drone record.
Operator → DroneOperator / EmergencyCoordinator / RescueTeamMember	—	Generalisation	Shared authentication behaviour, specialised responsibilities.
CommandCenter → RescueMission	1 : 1..*	Aggregation	The command center tracks missions but missions persist independently in the database.

7. Sequence / Interaction Diagram

The Interaction diagram depicted below describes the course of one such instance: a coordinator defines a mission, which allocates a drone; the drone travels to the destination and gathers information about it; the victim is identified; the alert goes through the notification system to one of the members of the rescue team, who provides the feedback. The above-mentioned interaction scenario was selected since it encompasses all three layers of the architecture (control, entity, and boundary), thus confirming the sufficiency of operations depicted in the class diagram.

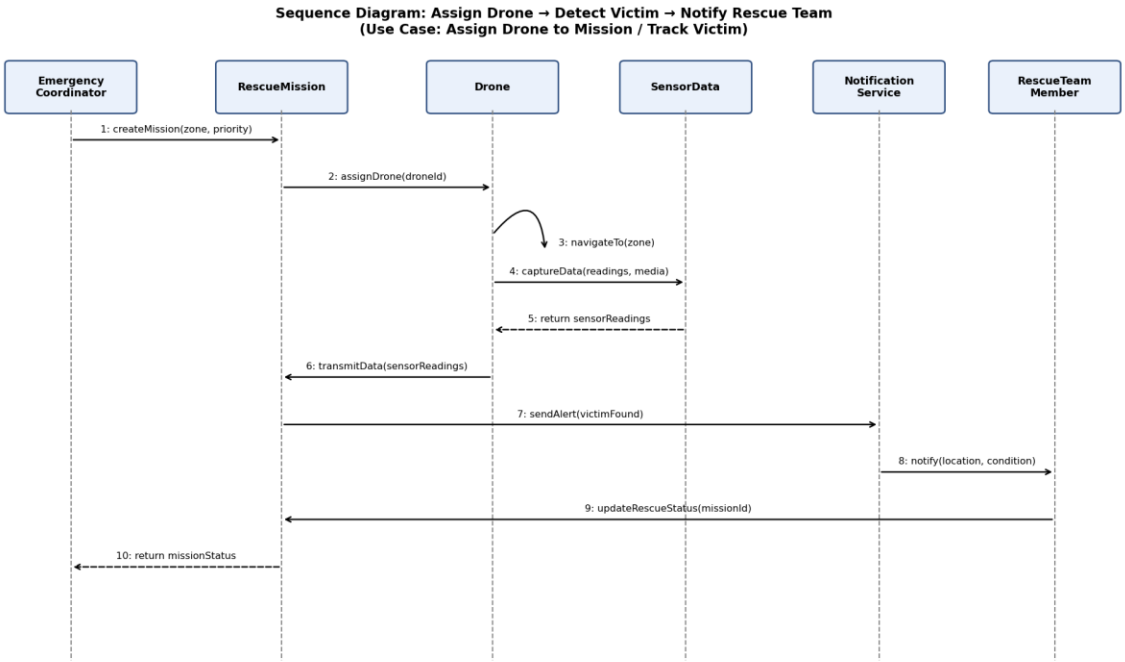

Figure 7.1 — Sequence Diagram: Assign Drone → Detect Victim → Notify Rescue Team

8. Activity Diagram

The activity diagram shows the process of execution of a mission by one drone from registering an incident up to generating a report, involving the decision making on availability of the drone, on generating an alarm due to detection, and on completion of the mission objective. The loop in "Mission objective met? – No" back to "Drone navigates autonomously" shows the repeated process of searching and capturing that forms the core of ADDRRS drones' functioning without continuous human intervention.

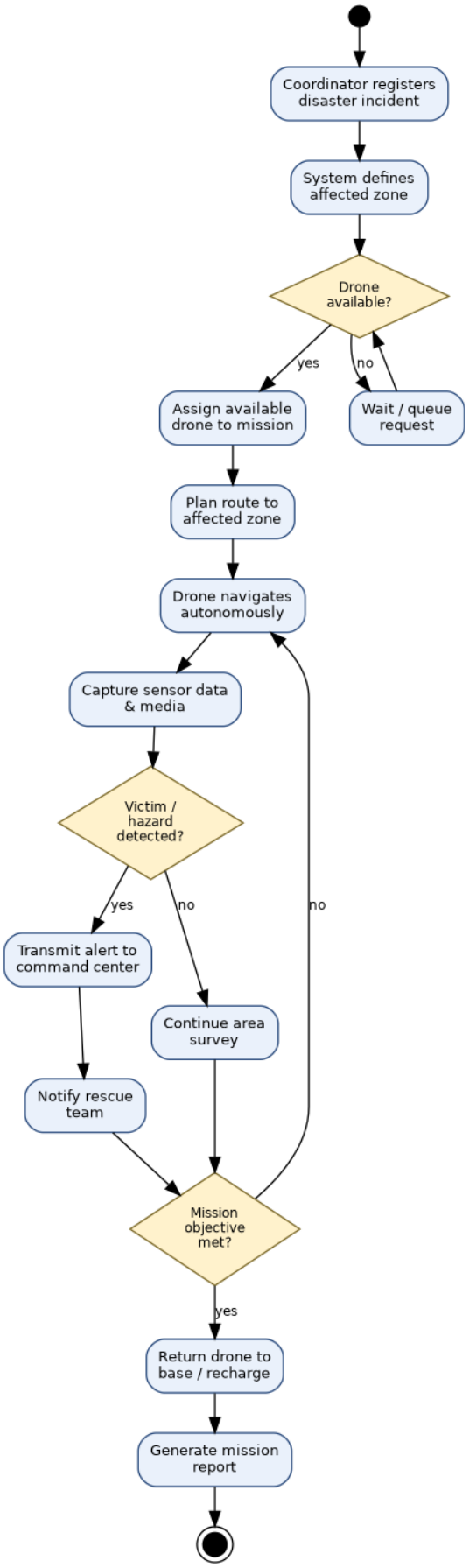
Coordinator registers disaster incident
System defines affected zone
Drone available?
yes
no
Assign available drone to mission
Wait / queue request
Plan route to affected zone
Drone navigates autonomously
Capture sensor data & media
Victim / hazard detected?
yes
no
no
Transmit alert to command center
Continue area survey
Notify rescue team
Mission objective met?
yes
Return drone to base / recharge
Generate mission report

Figure 8.1 — Activity Diagram: End-to-End Rescue Mission Workflow

9. State Transition Diagram

Due to the fact that the Drone class possesses the most operationally significant lifecycle in the system, it was selected for the state machine's prototyping. The transitions can be seen on the diagram below. The normal transition flow is Idle → Assigned → EnRoute → Surveying → DataTransmit → ReturningToBase → Charging → Idle. Additionally, two fault transitions (marked red) allow the drone to transition to a Fault state from EnRoute or Surveying if a malfunction is detected; from the Fault state, the drone will either trigger its fail-safe and return to base or, in case of a critical, unrecoverable failure, terminate the state machine altogether.

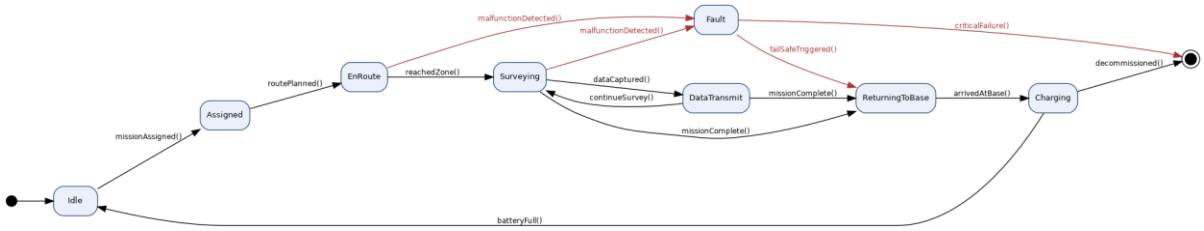

Figure 9.1 — State Transition Diagram for the Drone Class

10. Package Diagram

This system is structured into eight packages organized according to concerns, not according to the type of class. This ensures that each package can be tested independently and the modules dealing with the drone can be evolved without making any changes in the modules dealing with the coordinator. The direction of dependencies moves from Presentation to Persistence, downwards and outwards without any circular dependencies.

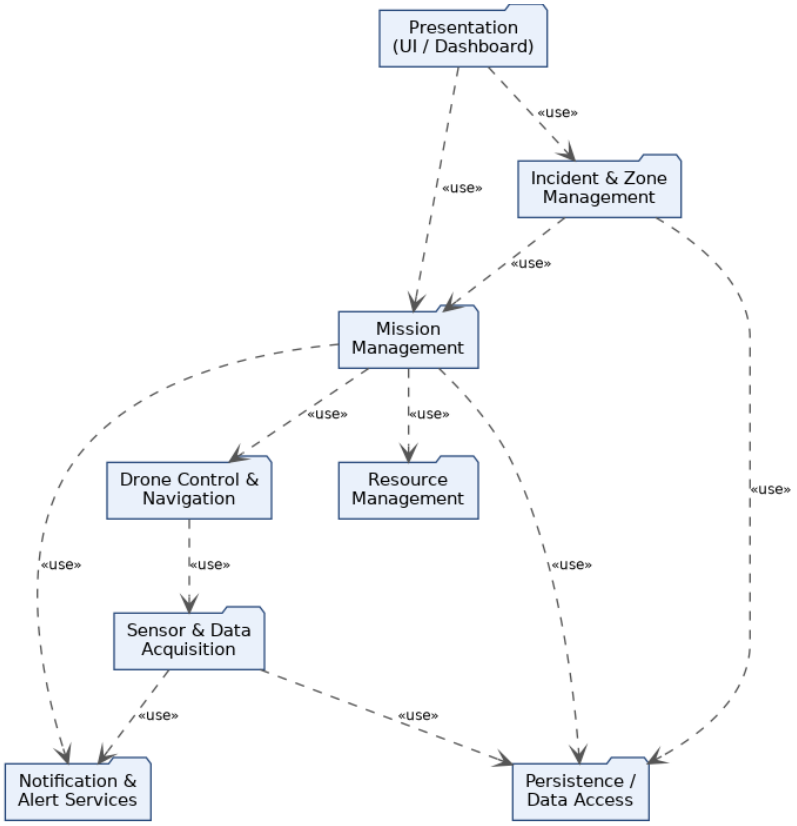

Figure 10.1 — Package Diagram for ADDRRS

11. Component Diagram

At the component level, each of the aforementioned package is realized by a single deployable component exposing a narrow interface towards its consumers. The Mission Management Service acts as the central orchestrator, only depending on Drone Control for navigation commands, on Resource Management for allocation, and on Data Access for persistence, but not on the Notification Component
for each operation directly, only the Sensor Data Processor publishes via IAlertPublish to avoid intertwining alerting logic with the mission orchestration code path.

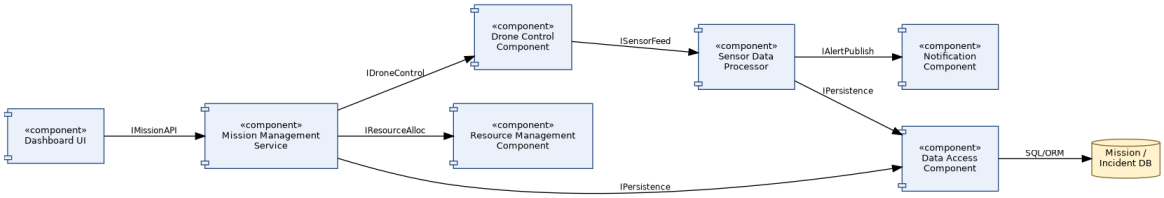

Figure 11.1 — Component Diagram for ADDRRS

12. Deployment Diagram

The deployment view separates four physical node types: the drone hardware itself (running embedded navigation/control firmware), a ground control workstation used by drone operators, a cloud application server hosting the mission and notification services behind a REST gateway, and a dedicated database server. Coordinators and rescue team members connect over HTTPS/REST from a browser or mobile app respectively, while drones and the ground station communicate with the cloud server over a wireless/4G-5G link — reflecting the requirement that drones operate autonomously in the field rather than tethered to a fixed control point.

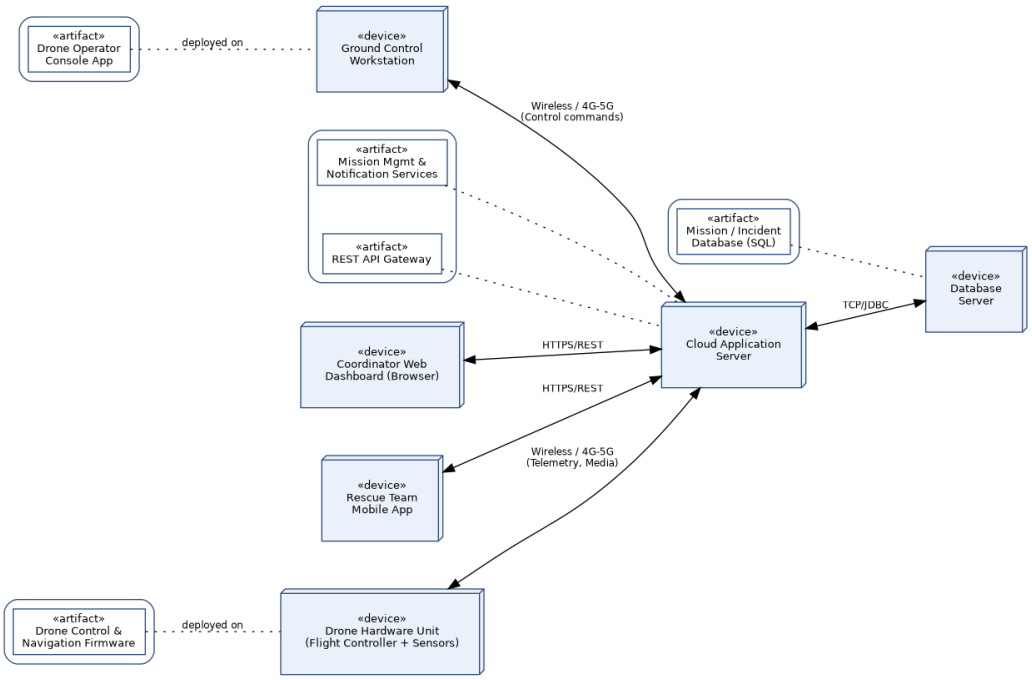

Figure 12.1 — Deployment Diagram for ADDRRS

13. Object Responsibility Analysis (CRC Cards)

Class-Responsibility-Collaborator analysis was used to sanity-check the class diagram: for each entity, the question asked was "what does this object know, and what does it do, without leaning on another object's internals?" This surfaced two adjustments during design — mission-report generation was pulled out of RescueMission into a light control step (avoiding a god-class), and route optimisation logic was kept inside Route rather than Drone, since a drone should not need to know how a path is computed, only how to follow one.

Class	Responsibilities	Collaborators
RescueMission	Own mission lifecycle state; coordinate drone and team assignment; request resource allocation.	Drone, RescueTeamMember, Resource, AffectedZone, CommandCenter
Drone	Track own status/battery/location; execute navigation and data capture; raise alert on fault.	Route, SensorData, Alert, RescueMission
SensorData	Hold one reading/media item with metadata; support transmission to command centre.	Drone, CommandCenter
Alert	Represent a single notification-worthy event; know its own severity/type; escalate if unacknowledged.	Drone, NotificationService, RescueTeamMember
Victim	Maintain current location and condition of a detected person; not responsible for notification delivery.	RescueTeamMember, Alert
Resource	Track allocation availability of one asset; know nothing about which mission logic decided to allocate it.	RescueMission

Class	Responsibilities	Collaborators
CommandCenter	Provide a single, consistent view across all active missions; delegate outward notification.	RescueMission, NotificationService
NotificationService	Translate an internal Alert into an outbound channel message; hide gateway-specific detail.	Alert, external SMS/push gateway

14. Design Axioms, Principles, and Design Pattern Justification

14.1 Design Axioms

Suh's two design axioms were applied deliberately rather than as an afterthought.

- Independence Axiom — each functional requirement was mapped to a design element that can be changed without side-effects on the others. For example, Route planning logic is isolated inside the Route class so that swapping the path-optimisation algorithm (e.g. shortest-path vs. obstacle-weighted) never touches Drone or RescueMission.
- Information Axiom — among designs that satisfy independence, the one with the least design information (simplest structure) was preferred. This is why NotificationService was modelled as a single boundary interface rather than one class per notification channel; the channel is a runtime parameter, not a class hierarchy, since no channel-specific behaviour beyond formatting is required.

14.2 SOLID / Cohesion and Coupling

Principle	Applied To	Justification
Single Responsibility	SensorData, Route	Each class owns exactly one concern — a reading, or a path — rather than mixing capture and transmission logic.
Open/Closed	Route.optimizeRoute()	New optimisation strategies can be added (Strategy pattern, below) without modifying Route's public interface.
Liskov Substitution	Operator hierarchy	DroneOperator, EmergencyCoordinator, and RescueTeamMember can all be used wherever an Operator is expected for login/logout.
Interface Segregation	NotificationService	Exposes only sendAlert()/sendUpdate() — consumers are not forced to depend on gateway configuration details.
Dependency Inversion	RescueMission → Drone	RescueMission depends on the Drone abstraction's public operations, not on drone hardware specifics.

High cohesion was achieved through concentrating each entity class's attributes and methods around a single real-world concept (a mission, a drone, a reading), while low coupling was achieved by routing cross-cutting concerns — persistence and notification — through dedicated boundary/control classes (DataAccess, NotificationService) rather than letting entity classes reach into each other's internals.

14.3 Design Patterns Applied

Pattern	Where Used	Reason
Singleton	CommandCenter	Exactly one command centre must exist to provide a consistent, system-wide view of active missions.
Observer	Alert → NotificationService → subscribers	Rescue teams and coordinators subscribe to alerts without CommandCenter needing to know who is listening.
Strategy	Route.optimizeRoute()	Lets different path-planning algorithms be swapped at runtime depending on terrain and battery constraints.
State	Drone lifecycle	Cleanly separates behaviour per lifecycle state (Idle, EnRoute, Fault, etc.) instead of large conditional blocks.
Factory Method	Resource creation	Centralises creation logic for different resource types (medical kit, vehicle, personnel) behind one creation interface.

15. Assumptions and Constraints

15.1 Assumptions

- Each drone comes with a GPS, at least one camera, and some environment sensors (temperature, gas) as standard hardware.
- Network coverage (4G/5G or satellite backup) will be available across all affected areas, at least intermittently, for telemetry and uploads.
- Coordinators, drone operators, and rescue team members will only be registered users; self-registration is outside the scope of this assignment.
- There will be a single command system instance serving one region/city at a time; federation across multiple regions is out of scope for this design.
- Victim detection via onboard sensors/images will have some classification logic (out of the scope of this assignment), but it should produce a confidence score that will be consumed by the Alert use case.

15.2 Constraints

- “The design must clearly identify system boundary, emergency actors, autonomous components, objects, classes, attributes, methods, and relationships, and all UML diagrams must remain consistent with the identified use cases and system behaviour.”
- “The model must explicitly account for real-time communication, autonomous operations, failure handling, security, reliability, scalability, and responsibility assignment.”
- “Design axioms and suitable design patterns must be applied wherever appropriate, and major design decisions must be justified.”
- “Regulatory airspace restrictions on autonomous drone flight (altitude ceilings, no-fly zones) apply and are treated as external constraints on Route planning rather than modelled in detail.”

16. Requirement-to-Use Case / Class Traceability

The table below demonstrates that every functional requirement traces forward to at least one use case and at least one class, closing the loop between analysis and design as required by the consistency constraint.

Req. ID	Requirement (short)	Use Case(s)	Class(es)
FR1	Register incident	Register Disaster Incident	DisasterIncident, Operator
FR2	Define affected zone	Define Affected Zone	AffectedZone, DisasterIncident
FR3	Assign drone to mission	Assign Drone to Mission	RescueMission, Drone
FR4	Plan flight route	Plan Route	Route, Drone
FR5	Monitor / override drone	Monitor Drone Status	Drone, DroneOperator
FR6	Capture & transmit sensor data	Collect Sensor Data, Transmit Media & Environmental Data	SensorData, Drone, CommandCenter
FR7	Raise detection alert	Handle Drone Failure/Alert, Track Victim	Alert, Victim, Drone
FR8	Notify rescue team	Send Emergency Notification	NotificationService, Alert
FR9	Update victim/rescue status	Track Victim, Manage Rescue Mission	Victim, RescueTeamMember, RescueMission
FR10	Allocate/release resources	Manage Resource	Resource, RescueMission
FR11	Handle drone fault / fail-safe	Handle Drone Failure/Alert	Drone, Alert
FR12	Generate mission report	Generate Mission Report	RescueMission, CommandCenter

17. Implementation / Prototype and Results

To validate that the class model is something more than a drawing, we wrote a small console-based Python prototype using all of the classes described in the class diagram (see ADDR_RS_prototype.py as an appendix), using the same scenario portrayed in Figure 7. This is a very simple prototype, useful only for executing the scenario and seeing that it runs without error (and of course for finding errors, if any).

17.1 Prototype Scope

- When creating an incident, a zone, and a mission, use the same attribute names as those shown in the class diagram.

- A simplified assignDrone() operation which selects idle drones on the basis of the battery threshold, replicating the main flow of the Assign Drone to Mission use case.

- A call to the function captureData() that is simulated in order to generate a sensor reading and randomly indicates a victim has been detected, thus illustrating the relationship between Track Victim and Send Emergency Notification.

• There is a basic state check for the Drone class that enforces the legal transitions as shown in the state diagram (Section 9) and rejects illegal ones such as the transition from Idle to DataTransmit.

17.2 Sample Run and Expected Results

When the prototype is run through a single simulated mission, the console displays output similar to that shown below: an incident and a zone are generated; a drone is selected from the available pool and is assigned; a route is planned using a placeholder list of way-points; a sensor reading is taken; if a detection is positive, then a Victim record and an Alert are created and a notification message is printed to represent the call to the Notification Service; and lastly a summary of the mission report is printed together with placeholders for the mission duration, the number of detections, and the resource usage. This shows that the responsibilities given in Section 13 are enough to carry out the primary scenario end to end without it being necessary to add any hidden logic to any class.

If you are carrying out a full production implementation, the following steps should be taken: to substitute the console I/O with a REST API layer (this being in accordance with the Component Diagram's IMissionAPI/IDroneControl interfaces), to connect DataAccess to an actual relational database as indicated in the Deployment Diagram, and to replace the stochastic detection stub with a real onboard image-classification model.

18. Reflection on Design Decisions and Learning Outcomes

Carrying out this assignment helped to clarify a distinction which is easy to describe but difficult to apply in a consistent way: the difference between an object that owns data and one that owns a decision. In the first versions of the class diagram, the route optimisation logic was placed directly on the Drone, a arrangement that seemed natural since a drone 'does the flying', but it made the Drone class responsible for both the hardware state and the path-planning strategy — two aspects which change for entirely different reasons. The one change that had the greatest effect on improving the design's independence was to move that logic into Route and then to apply the Strategy pattern to it, a change which resulted directly from carrying out the CRC-card exercise in Section 13 rather than from relying solely on the diagram.

The hardest aspect of the design was finding a way to combine autonomy with accountability: although the specification calls for the drone to function as a system actor and to generate its own alerts, each alert must still be traceable to a decision that can be reviewed by a human, which is the reason why Alert and Notification Service were kept as separate classes rather than incorporating the notification function directly into Drone. Similarly, when selecting the UML views a compromise had to be made – the state diagram was centred on the Drone rather than on the Rescue Mission since the drone's failure modes are the most operationally dangerous element of the system and so are the most important to model explicitly, even though a state machine at the mission level could also have been argued for.

For the Sustainable Development Goals relevant to this assignment — SDG 3, 9, 11, and 13 — the relationship is direct not merely symbolic: achieving faster and more systematic location of victims in a disaster (SDG 3) relies on resilient and well-instrumented infrastructure (SDG 9) which can be deployed in cities and settlements that are at risk (SDG 11), and the whole reason for having autonomous, rapidly responsive equipment is that climate-related disasters (SDG 13) are becoming more frequent and severe, which makes manual-only rescue coordination increasingly inadequate. The exercise as a whole demonstrated that the OOAD approach — carrying out actor and use-case analysis before class design and then carrying out a CRC review before finalising

the relationships — results in a model that is not only diagrammatically complete but also genuinely traceable from the requirements through to the actual code.